A good fit is a **Customer Support Ticket Triage & Resolution Agent**.

It is simple enough to build with a small amount of code/data, but it naturally lets you demonstrate almost all of the important LangChain/LangGraph/LangSmith agentic concepts without turning into a large project.

1. Use case

Build an agent that receives a customer support ticket such as:

> "I was charged twice for my subscription, and I need the extra charge refunded."

The system should:

1. Understand/classify the ticket.
2. Gather relevant information using tools.
3. Have specialized agents investigate different aspects.
4. Execute some investigations in parallel.
5. Reflect/review its proposed resolution.
6. Ask a human for approval when the action is sensitive.
7. Produce a concise final response.
8. Trace and evaluate the entire workflow using LangSmith.

You can keep the underlying data extremely simple—SQLite or even JSON files.

---

2. Why this is a good learning project

The workflow can expose these concepts:

Agentic concept	Where you'll use it
LangChain	LLMs, prompts, tools, structured output
LangGraph	Orchestration/state machine
Tools	Customer/order/payment lookup
Agent	Ticket investigation
Multi-agent	Billing + account + policy specialists
Parallel execution	Investigate multiple dimensions simultaneously
Reflection	Review proposed resolution
Human-in-the-loop	Approve refund/escalation

Agentic concept	Where you'll use it
Summarization	Generate final support response
Conditional routing	Decide whether human approval is required
State	Maintain ticket context throughout graph
LangSmith	Tracing, debugging, evaluation
Structured output	Consistent agent results
Retry/error handling	Recover from failed tool/agent calls

The important thing is that **you don't need a sophisticated business system** to demonstrate these concepts.

---

3. Keep the domain deliberately small

Create a tiny fictional company, for example:

AcmeCloud — a SaaS subscription company.

Create only three types of data:

Customers

```
customer_id
name
subscription
account_status
```

Payments

```
payment_id
customer_id
amount
date
status
```

Support policies

```
policy_id
category
policy_text
```

For example:

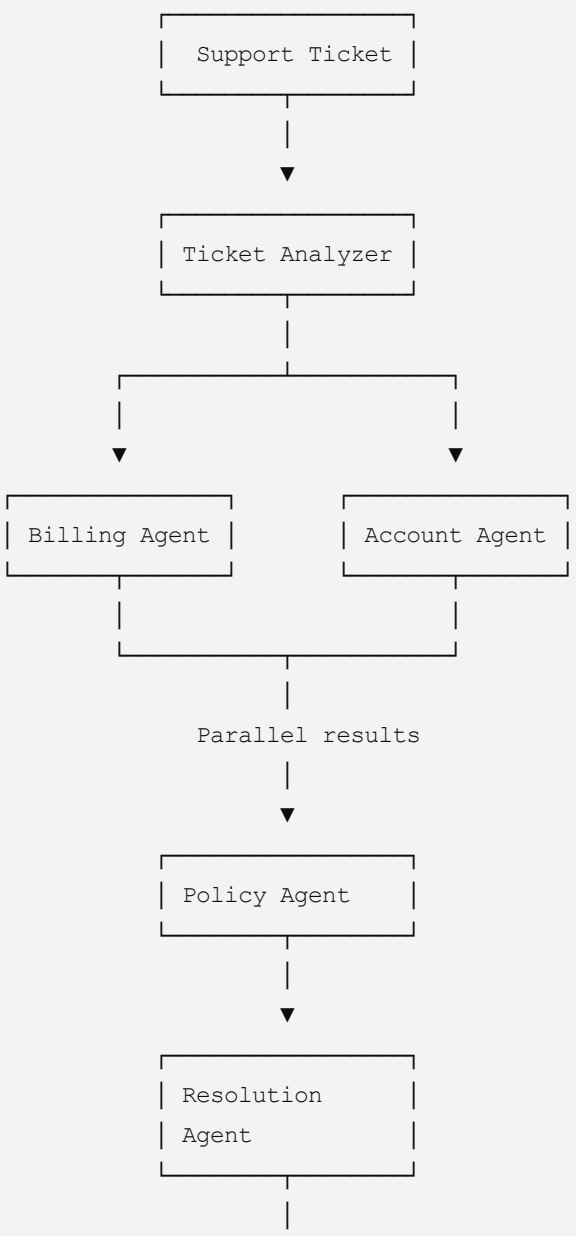
- Duplicate payment → eligible for automatic refund
- Refund > $100 → human approval required
- Account cancellation → human approval required
- Normal informational requests → no approval

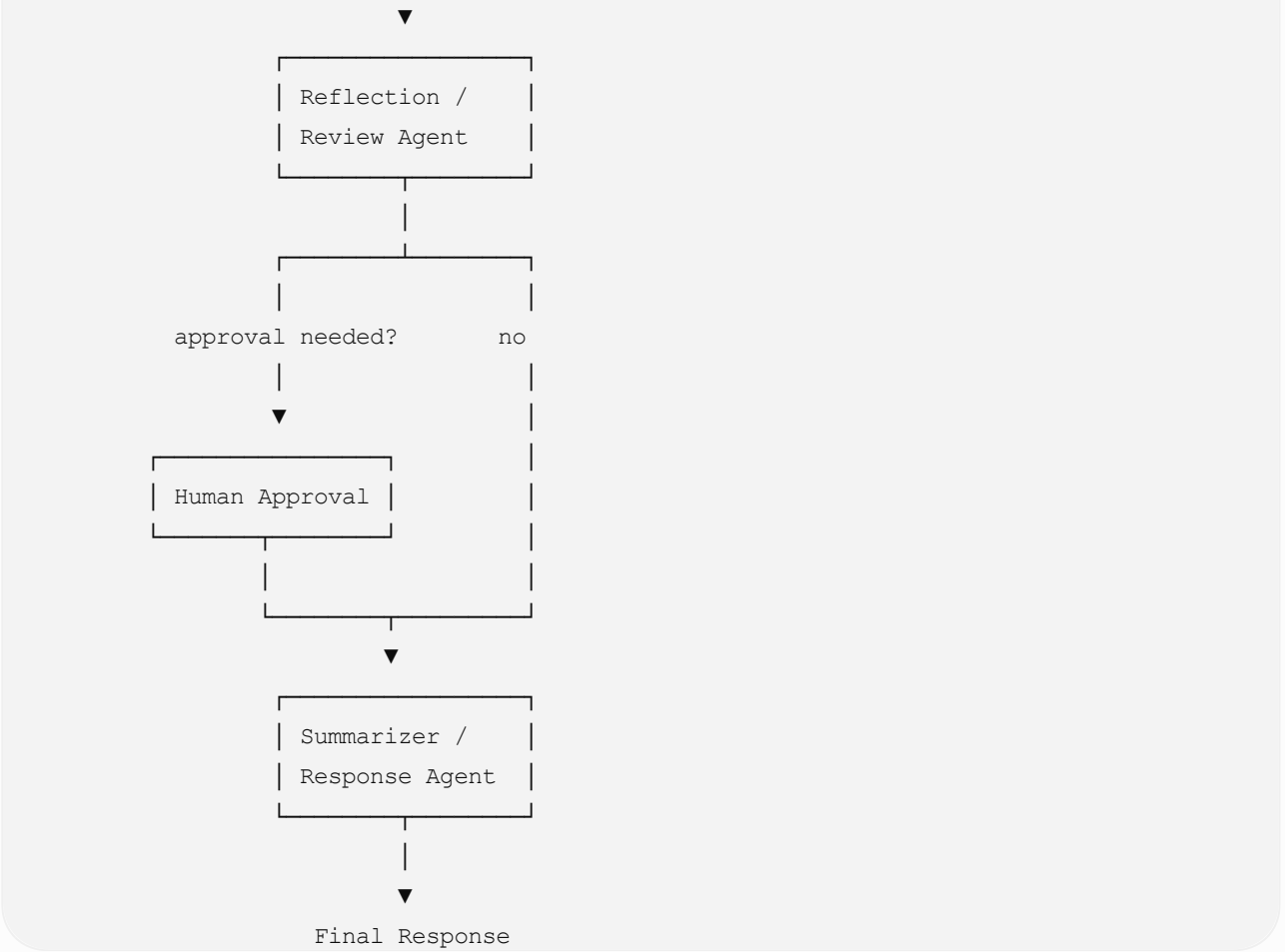
You don't need a real database initially. SQLite is enough.

---

4. Overall architecture

The graph could look like this:





This is small enough to implement, while still looking like a genuine agentic workflow.

---

5. Phase 1 — Build the basic LangChain application

Start **without LangGraph**.

The goal is to understand the individual components first.

Step 1 — Select an LLM

Use whichever provider you already have convenient access to.

For example:

- OpenAI-compatible API
- Groq
- Mistral
- Ollama
- another provider supported by LangChain

Don't spend time optimizing the model.

A relatively inexpensive model is sufficient.

Step 2 — Create the ticket structure

Define a simple ticket:

```
ticket_id
customer_id
message
```

Example:

```
Ticket: 1001
Customer: C123
Message:
"I was charged twice for my subscription.
Please refund the duplicate payment."
```

Step 3 — Create the initial ticket analyzer

Have the LLM determine:

```
category
urgency
customer_intent
requires_investigation
```

Possible categories:

```
billing
account
technical
general
```

Use structured output rather than asking the LLM to return arbitrary text.

This gives you your first lesson in **reliable LLM outputs**.

---

6. Phase 2 — Introduce tools

Create 3–4 very simple tools.

Tool 1 — Customer lookup

Input:

```
customer_id
```

Returns:

```
customer information
subscription information
account status
```

Tool 2 — Payment lookup

Input:

```
customer_id
```

Returns recent payments.

Tool 3 — Policy lookup

Input:

```
category
```

Returns applicable support policies.

Tool 4 — Refund eligibility

Input:

```
payment
policy
```

Returns:

```
eligible
reason
requires_human_approval
```

---

7. Phase 3 — Build your first agent

Create a simple **Billing Agent**.

Its job:

1. Read the ticket.
2. Determine what information it needs.
3. Call the appropriate tools.

4. Analyze the results.
5. Produce a structured investigation result.

For example:

```
Billing investigation:

Duplicate payment detected: YES
Original payment: $49.99
Duplicate payment: $49.99
Refund eligible: YES
Human approval required: NO
Recommendation: Refund payment P456
```

This gives you your first real **tool-using agent**.

---

8. Phase 4 — Move orchestration into LangGraph

Now introduce LangGraph.

Don't immediately create a complex graph.

Start with:

```
START
  ↓
Analyze Ticket
  ↓
Billing Agent
  ↓
Resolution
  ↓
END
```

Create a graph state containing things such as:

```
ticket
ticket_analysis
investigation_results
policy_results
resolution
review
human_decision
final_response
```

The important learning objective here is:

> **The graph state is the shared memory/context of the workflow.**

Each node reads from and writes to this state.

---

9. Phase 5 — Add multi-agent behavior

Now introduce two specialized agents.

Billing Agent

Responsible for:

- payments
- invoices
- refunds
- billing issues

Account Agent

Responsible for:

- subscriptions
- account status
- cancellation
- account information

You could eventually add:

Technical Agent

Responsible for:

- technical problems
- service status
- troubleshooting

But I would initially implement only **Billing + Account**.

This keeps the project small.

---

10. Phase 6 — Add parallel execution

This is one of the most valuable LangGraph concepts to learn.

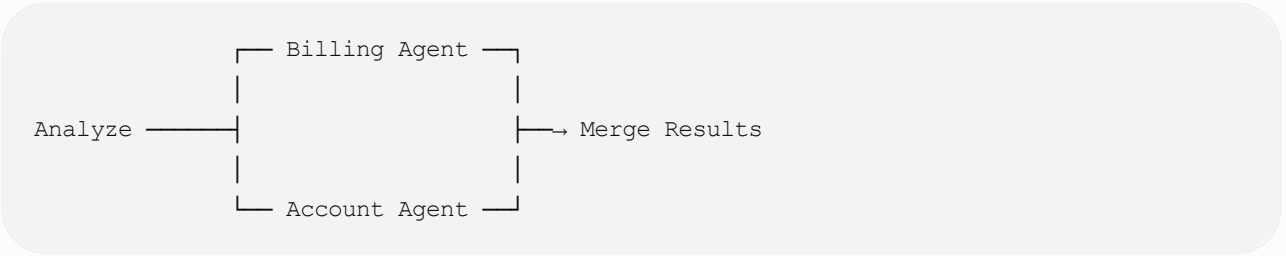
Suppose a ticket says:

> "I was charged twice and my subscription still shows as inactive."

The graph should not do:

```
Billing Agent
     ↓
Account Agent
```

Instead:



Both agents execute independently.

Then a subsequent node combines their results.

Why this is useful

You learn:

- parallel branches
- shared graph state
- synchronization
- merging results
- concurrency
- independent agent responsibilities

This is a much more meaningful LangGraph exercise than simply creating a linear chain.

---

11. Phase 7 — Add a Policy Agent

After the specialist agents finish, send their results to a **Policy Agent**.

Its job is not to investigate the customer.

Its job is to answer:

> "Given the investigation results and company policies, what actions are allowed?"

For example:

```
Investigation:
Duplicate charge = true
Amount = $49.99

Policy:
Duplicate charges may be refunded.

Decision:
Refund allowed.
Human approval not required.
```

This introduces an important architectural distinction:

Investigation ≠ decision.

---

12. Phase 8 — Add a Resolution Agent

Now create a Resolution Agent.

Input:

- original ticket
- ticket classification
- billing investigation
- account investigation
- policy decision

Output:

```
resolution_type
action
reason
requires_human_approval
confidence
customer_response_summary
```

For example:

```
resolution_type: refund
action: refund payment P456
reason: duplicate charge
requires_human_approval: false
confidence: 0.94
```

At this point you have a complete agentic decision pipeline.

---

13. Phase 9 — Add reflection

Now introduce one of the most important agentic patterns:

Generate → Critique → Revise

Instead of trusting the Resolution Agent immediately:

```
Resolution Agent
       ↓
Reflection Agent
       ↓
Is resolution correct?
       ↓
   Yes / No
```

The Reflection Agent should check:

1. Did the agents correctly understand the ticket?
2. Is the proposed action supported by evidence?
3. Does it comply with policy?
4. Is there missing information?
5. Is human approval required?
6. Is the confidence reasonable?

Return something like:

```
approved: false

issues:
- Refund amount was not verified against payment record.

recommendation:
- Re-run payment lookup before approving refund.
```

---

14. Add a reflection loop

This is where LangGraph becomes particularly interesting.

Instead of:

```
Resolution → Reflection → END
```

implement:

But put a **maximum iteration count**, such as 2–3.

This prevents an infinite agent loop.

For your first implementation, I'd use:

maximum 2 reflection cycles.

That is enough to demonstrate the concept.

---

15. Phase 10 — Human-in-the-loop

Now introduce the human.

Create rules such as:

```
Refund <= $100
    → automatic

Refund > $100
    → human approval

Account cancellation
    → human approval
```

Your graph becomes:

```
Resolution
    ↓
```

```
Reflection
    ↓
Approval Required?
   / \
 No   Yes
 |     |
 |     ▼
 |   Human
 |   Approval
 |     |
 └──┬──┘
    ▼
 Final Response
```

The important point is that **the graph pauses** waiting for the human decision.

The human can choose:

```
Approve
Reject
Request more investigation
```

This is much more educational than simply asking the user a question inside an LLM prompt.

---

16. Phase 11 — Add summarization

Finally, create a dedicated **Response/Summarization Agent**.

It receives all relevant information but produces only the customer-facing answer.

For example:

> "We've identified a duplicate charge of $49.99 on your account. The duplicate payment is eligible for a refund, and the refund has been approved."

The summarizer should **not perform investigation**.

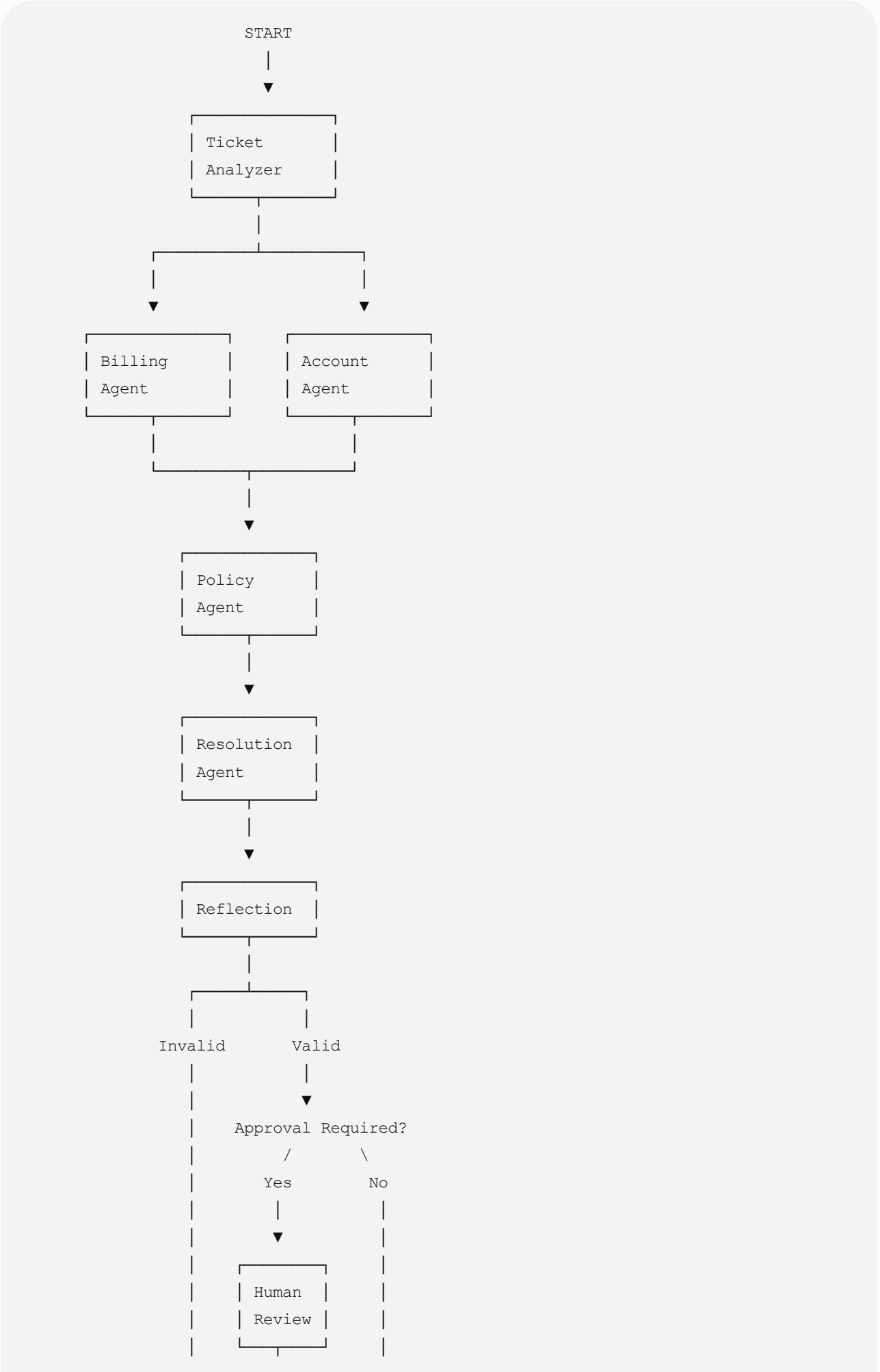
This teaches an important principle:

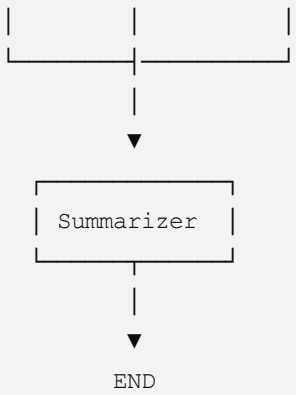
> Separate reasoning/workflow state from the final user-facing output.

---

17. Final graph

Your finished graph should approximately look like this:





---

18. Phase 12 — LangSmith integration

Don't wait until the end to add LangSmith.

Connect LangSmith from the beginning.

You want to observe:

```
Ticket
 └── Ticket Analyzer
      ├── Billing Agent
      │    ├── payment_lookup
      │    └── customer_lookup
      │
      ├── Account Agent
      │    └── customer_lookup
      │
      ├── Policy Agent
      │    └── policy_lookup
      │
      ├── Resolution Agent
      │
      ├── Reflection Agent
      │
      └── Summarizer
```

This will let you see exactly how an agentic workflow differs from a simple LLM chain.

---

19. Add LangSmith evaluation

Once the workflow works, create a small evaluation dataset.

You only need around **15–20 tickets**.

For example:

Ticket	Expected category	Expected action
Duplicate payment	Billing	Refund
Password forgotten	Account	Password reset
Subscription inactive	Account	Investigate
$500 duplicate payment	Billing	Human approval
Cancel subscription	Account	Human approval
"How do I change my email?"	Account	Instructions

Evaluate:

Classification accuracy

Did the Ticket Analyzer identify the right category?

Tool selection

Did the agent call the appropriate tool?

Resolution correctness

Was the correct action selected?

Policy compliance

Did it follow the business rules?

Final answer quality

Was the customer response accurate and concise?

This gives you practical exposure to **LLM/agent evaluation**, rather than only tracing.

---

20. Recommended implementation sequence

I would **not** implement everything at once.

Build it incrementally:

Iteration 1 — Basic LangChain

- ☐ LLM

- [ ] Ticket prompt
- [ ] Structured ticket classification

Iteration 2 — Tools

- [ ] Customer lookup
- [ ] Payment lookup
- [ ] Policy lookup

Iteration 3 — First agent

- [ ] Billing Agent
- [ ] Tool calling
- [ ] Structured investigation result

Iteration 4 — LangGraph

- [ ] Graph state
- [ ] Nodes
- [ ] Conditional edges
- [ ] Basic end-to-end workflow

Iteration 5 — Multi-agent

- [ ] Billing Agent
- [ ] Account Agent
- [ ] Result aggregation

Iteration 6 — Parallelism

- [ ] Execute Billing + Account agents concurrently
- [ ] Merge results

Iteration 7 — Decision making

- [ ] Policy Agent
- [ ] Resolution Agent

Iteration 8 — Reflection

- [ ] Reflection Agent
- [ ] Validation
- [ ] Retry/loop
- [ ] Maximum iteration limit

Iteration 9 — Human-in-the-loop

- [ ] Determine whether approval is required
- [ ] Pause graph

- [ ] Human decision
- [ ] Resume graph

Iteration 10 — Final response

- [ ] Summarization/response agent
- [ ] Customer-facing output

Iteration 11 — LangSmith

- [ ] Tracing
- [ ] Inspect traces
- [ ] Create dataset
- [ ] Run evaluations
- [ ] Compare iterations

---

21. Keep the first version deliberately constrained

I would **avoid** adding these initially:

- RAG
- vector databases
- web search
- long-term memory
- complicated APIs
- authentication
- frontend
- real payment systems
- sophisticated planning
- dozens of tools

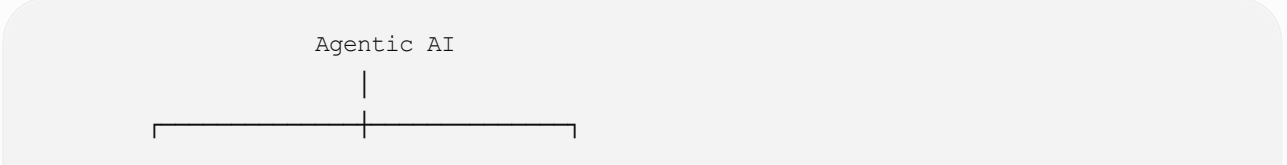
They would distract from the concepts you're trying to learn.

You can add them later.

---

22. What you'll learn from this one project

By the time you finish, you will have touched almost the entire practical LangGraph/LangChain agentic stack:



```
   |             |             |
LangChain     LangGraph     LangSmith
   |             |             |
LLMs          State/Edges     Tracing
Tools         Routing         Evaluation
Prompts       Parallelism     Datasets
Structured    Loops           Experiments
Output        HITL
              Multi-agent
              Reflection
```

And, importantly, the **business logic is simple enough that you can focus on the agent architecture rather than spending days building the application itself**.

My recommended scope

If your primary goal is **learning LangChain + LangGraph + LangSmith**, I'd target roughly **1–2 days for the basic implementation**, then another **1–2 days for reflection, HITL, tracing and evaluation**.

The particularly valuable progression is:

single agent → tools → LangGraph → multi-agent → parallel agents → reflection loop → human approval → summarization → LangSmith evaluation.

That progression gives you a very clean learning path from simple LLM application to a genuinely agentic workflow.